German University in Cairo
Department of Computer Science
Assoc. Prof. Mervat Abu El-Kheir

Architecture of Massively Scalable Applications, Spring 2026

Milestone 1: Ride Hailing (Uber) Platform

Core Services Foundation Deadline is Saturday 11/04/2026 at 11:59 PM

Contents

1 Overview

This milestone is worth **15%** of your final grade. You will build a **Ride Hailing Platform** consisting of 5 independently runnable Spring Boot services organized as a **Maven multi-module project**. All 5 services connect to a single shared PostgreSQL database and are orchestrated via a single `docker-compose.yaml`.

In future milestones, these services will be refactored into true microservices with separate databases, inter-service communication (OpenFeign, RabbitMQ, etc.), and Kubernetes deployment. For now, they are independent applications sharing one database.

Technologies: Spring Boot, PostgreSQL, Spring Data JPA, Docker, Docker Compose, Maven multi-module, Git & GitHub.

Services:

a) **User Service** (port 8081) — Rider accounts, profiles, preferences

b) **Driver Service** (port 8082) — Driver profiles, vehicles, availability

c) **Ride Service** (port 8083) — Ride lifecycle, fare calculation, ride history

d) **Location Service** (port 8084) — Driver GPS tracking, proximity queries

e) **Payment Service** (port 8085) — Payment processing, refunds, revenue

Deliverables: 45 features (9 per service), full CRUD for all entities, Dockerized multi-service application, Git history with personalized feature branches and PRs.

2 Personalization & Team Roster

Every team member's work must be **individually identifiable** in Git history. The auto-grader cross-references the `team.json` roster against `git log` to determine who built what. Violations result in **zero credit** for the affected member.

2.1 team.json (Required)

Create a file named `team.json` in the **project root directory** (next to the parent `pom.xml`). It contains a JSON array with one entry per team member:

```
[{"studentId":"55-8078","name":"Ahmed
Ali","githubUsername":"ahmed-ali-dev","service":"user-service"}, ...]
```

Fields per entry:

- `studentId` — University ID in the format `XX-XXXX`
- `name` — Full name
- `githubUsername` — The exact GitHub username that will author commits
- `service` — Which service module this member belongs to (one of: `user-service`, `driver-service`, `ride-service`, `location-service`, `payment-service`)

The auto-grader reads this file and then runs specific tests to find each member's commits. It matches those commits against branch names to determine which features each member implemented. **Do not self-declare features** — the grader discovers them automatically.

2.2 Branch Naming (Mandatory)

All feature branches must include the implementing member's student ID:

`feat/<service>/<feature-name>/<studentId>`

Example: student `55-8078` implementing S1-F1 (Service 1, Feature 1) creates:

`feat/user/S1-F1/55-8078`

2.3 Commit Message Format (Mandatory)

All commits must follow this convention:

`feat(<service name>): <description> (ID)`

Examples: `feat(user-service): add User entity model (55-8078)`, `fix(ride-service): fix null handling in search query (55-8078)`.

The auto-grader matches each commit's Git author (from `team.json`) against the student ID in the message. Mismatched or missing IDs will result in failures during grading. So even if you named the commit message wrong or the branch name was wrong and did follow the criteria, you know how to fix it using the commands that you took in the Git Lab.

3 Project Structure — Multi-Module Maven

The project is organized as a Maven multi-module project. The root directory contains a **parent POM** with `<packaging>pom</packaging>` that declares all 5 services as modules. Each service is a fully independent Spring Boot application with its own POM, source tree, and `@SpringBootApplication` entry point. Section 4 provides full step-by-step instructions for creating this structure.

3.1 Port & Database Configuration

All services connect to the **same PostgreSQL database** (`uberdb`). Each runs on a different port:

Service	Internal Port	Docker Host Port
user-service	8080	8081
driver-service	8080	8082
ride-service	8080	8083
location-service	8080	8084
payment-service	8080	8085

Each service's `application.properties` sets `server.port=8080`. The port differentiation happens in `docker-compose.yaml` via host-port mapping (e.g., `8081:8080` for user-service). When running services locally without Docker during development, each developer works on their own service and can use port 8080 since they only run one service at a time.

Each service configures: `spring.datasource.url=jdbc:postgresql://localhost:5432/uberdb` (or `postgres:5432` inside Docker), `ddl-auto=update`, and `show-sql=true`.

4 Step-by-Step: Creating the Multi-Module Project

This section walks you through creating the entire project structure from scratch. Two approaches are provided: **Option A** using IntelliJ IDEA (recommended), and **Option B** using Spring Initializr for

students not using IntelliJ. Both produce the same result. Follow **one** of the two options, then continue with the shared steps.

Important architectural note: The root project acts as an **aggregator only**. It does not contain source code and does not act as a Maven parent for the child modules. Each service module keeps its own generated Spring Boot parent. The root `pom.xml` simply lists the modules so you can build them all with a single command.

4.1 Using IntelliJ IDEA (Recommended)

4.1.1 A1. Create the Parent Project

a) Open IntelliJ IDEA → **File** → **New** → **Project**.

b) On the left panel, select **Java**.

c) On the right, set **Build system** to **Maven**.

d) Set the **JDK** to **25**.

e) Uncheck **Add sample code**.

f) Expand **Advanced Settings** (at the bottom of the dialog).

g) Fill in:

- **Name:** `uber`
- **GroupId:** `com.teamXX.uber` (replace `XX` with your team number)
- **ArtifactId:** `uber`

h) Click **Create**.

i) IntelliJ creates a Maven project with a `pom.xml`. Open this `pom.xml` and manually add the following line inside the `<project>` block (after `<version>`):

```
<packaging>pom</packaging>
```

j) If IntelliJ created a `src/` folder in the root project, **delete it** — the root aggregator project has no source code.

k) **Reload Maven:** Click the Maven reload icon (circular arrows) in the Maven tool window, or right-click the `pom.xml` → **Maven** → **Reload project**.

4.1.2 A2. Add Each Service as a Module

Repeat the following for each of the 5 services (`user-service`, `driver-service`, `ride-service`, `location-service`, `payment-service`):

a) Right-click the project root (`uber`) in the Project panel → **New** → **Module**.

b) Select **Spring Boot** on the left.

c) Configure the service information:

- **Name / Artifact:** the service name (e.g., `user-service`)
- **Group:** `com.teamXX.uber`
- **Package name:** Use the service domain name **without hyphens**:
  - `com.teamXX.uber.user`
  - `com.teamXX.uber.driver`

- `com.teamXX.uber.ride`
- `com.teamXX.uber.location`
- `com.teamXX.uber.payment`

- **Type / Build system:** Maven
- **Java:** 25
- **Packaging:** Jar

Warning: The `artifactId` can use hyphens (e.g., `user-service`), but **Java package names cannot contain hyphens**. Use `.user`, not `.user-service`, in the package name.

d) Click **Next** to continue to the dependencies page.

e) Select these dependencies: **Spring Web**, **Spring Boot DevTools**, **Spring Data JPA**, **PostgreSQL Driver**, **Rest Repositories**.

f) Click **Create**.

4.1.3 A3. What IntelliJ Does (and Does Not Do) Automatically

After creating each service module, IntelliJ will usually:

- Create the service folder with its own `pom.xml` and `src/` tree.
- Generate the `@SpringBootApplication` main class.

However, IntelliJ may **not** automatically add the `<module>` entry to the root `pom.xml`. You must verify this yourself after each module creation.

4.1.4 A4. Verify and Complete the Root POM

After creating all 5 service modules:

a) Open the root `uber/pom.xml`.

b) Confirm that `<packaging>pom</packaging>` exists.

c) Check whether all 5 services appear inside a `<modules>` block. If **any are missing**, add them manually:

```
<modules>
    <module>user-service</module>
    <module>driver-service</module>
    <module>ride-service</module>
    <module>location-service</module>
    <module>payment-service</module>
</modules>
```

d) Save the file.

e) **Reload Maven** to pick up the changes.

f) Confirm there are no Maven errors in the tool window.

4.1.5 A5. Important: Do NOT Modify Child Service POMs

Each generated service already has its own `<parent>` block pointing to Spring Boot. **Do not replace or modify this parent.** The root `uber` POM acts only as an **aggregator** (it lists the modules so you can build them all at once). It is **not** a Maven parent for the child modules.

In other words:

- **Do not** add a `<parent>` block pointing to `uber` inside any child POM.
- **Do not** remove the existing Spring Boot `<parent>` from any child POM.
- The only connection between the root POM and the children is the `<modules>` block in the root.

You are done with Option A. Skip to *Shared Steps (Both Options)* below.

4.1.6 Add jackson-databind Dependency

In each service's `pom.xml`, add the following dependency inside the `<dependencies>` block (required for Hibernate JSONB support):

```
<dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
</dependency>
```

After adding, **reload Maven**.

4.1.7 Configure application.properties per Service

Inside each service's `src/main/resources/application.properties`, add:

```
server.port=8080

spring.datasource.url=jdbc:postgresql://localhost:5432/uberdb
spring.datasource.username=postgres
spring.datasource.password=postgres
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=
                org.hibernate.dialect.PostgreSQLDialect
```

All services use port 8080 internally. The port differentiation to 8081–8085 happens in `docker-compose.yaml` (covered in the Dockerization section).

4.1.8 Create Sub-Packages

Inside each service's main package (e.g., `src/main/java/com/teamXX/uber/user/`), create these sub-packages: `model/`, `repository/`, `service/`, `controller/`, `dto/`.

4.1.9 Add a Health Endpoint to Each Service

In each service's `controller/` package, create a simple health controller that exposes a GET endpoint returning `"OK"` (200):

- User Service: `GET /api/users/health`
- Driver Service: `GET /api/drivers/health`

- Ride Service: `GET /api/rides/health`
- Location Service: `GET /api/locations/health`
- Payment Service: `GET /api/payments/health`

4.1.10 Create team.json

In the project root (next to the root `pom.xml`), create `team.json` as described in Section 2.

4.1.11 Verify the Build

Start PostgreSQL via Docker:

```
docker compose up -d
```

(This requires the `docker-compose.yaml` with just the PostgreSQL service — see Section 6.)
Then build all services from the project root:

```
mvn clean package -DskipTests
```

If the build succeeds, you should see `BUILD SUCCESS` and a JAR in each service's `target/` directory.
To test a single service locally:

```
cd user-service
mvn spring-boot:run
```

Then visit `http://localhost:8080/api/users/health` to confirm it responds with `"OK"`.

4.2 Final Project Structure

After completing all steps, your project should look like this:

```
uber/
+-- pom.xml                              (root aggregator POM, packaging=pom)
+-- team.json                            (team roster)
+-- docker-compose.yaml                  (PostgreSQL, later: all services)
+-- .gitignore
+-- user-service/
|   +-- pom.xml                          (own Spring Boot parent, NOT uber)
|   +-- Dockerfile                       (added in Phase D)
|   +-- src/main/java/com/teamXX/uber/user/
|       +-- UserApplication.java         (@SpringBootApplication)
|       +-- model/
|       +-- repository/
|       +-- service/
|       +-- controller/
|       +-- dto/
+-- driver-service/
|   +-- (same structure)
+-- ride-service/
|   +-- (same structure)
+-- location-service/
|   +-- (same structure)
+-- payment-service/
    +-- (same structure)
```

5 Git Workflow — Initial Setup

a) Initialize a Git repository in the project root.

b) Create a `.gitignore` excluding: `target/`, `.idea/`, `*.iml`, `.env`, `*.class`, `.DS_Store`.

c) Create the parent POM, all 5 child POM files, the package structure, and the `team.json` file.

d) Make an initial commit: `init: project setup by (Name_ID)` (e.g: init: project setup by (Mohamed_Ayman_52_8078)) (note: this is supposed to be the only commit message that will not follow the commit message rule that is stated in this description).

e) Create a **private** GitHub repository named `TeamNumber-TeamName-Uber` (e.g: 40-Random-Uber).

f) Push to remote and rename the default branch to `main`.

g) **Branch protection:** Enable *"Require a pull request before merging"* on `main`. (The rule may not be enforced on the free version of git so you have to enforce it manually as the grader will check the generated PRs)

h) Add `Scalable-Submissions` as a collaborator.

i) **Each member** must contribute to the project through Git using their own GitHub account. The auto-grader verifies all GitHub usernames from `team.json` appear in history of the repo. ***If someone didn't contribute to the project (no-commits for him/her), will be considered as if they didn't work at all in the milestone and will receive a ZERO as a result in the milestone grade.***

6 Database Setup (Docker Compose)

The `docker-compose.yaml` in the project root contains a PostgreSQL service with:

- Container name: `uber-db`
- Port mapping: `5432:5432`
- Environment variables: user `postgres`, password `postgres`, database `uberdb`
- Named volume `pgdata` for data persistence

In Phase D (Dockerization), you will add all 5 application services to this same file.

Verify: Run `docker compose up -d` (PostgreSQL only at first), start any service locally, and confirm it connects to the database.

7 Entity Models

All entities use auto-generated `Long` IDs. Some entities include one JSONB column implemented as a `Map<String, Object>` using the Hibernate JSONB annotations learned in Lab 4. Since all services share one database, tables from different services coexist in the same database schema.

Each service defines JPA relationships (`@OneToMany`, `@ManyToOne`, join entities) **between entities within the same service only**. References to entities in other services are stored as plain `Long` foreign key columns (not JPA-managed relationships), and cross-service data access uses native SQL JOINs on the shared database. Remember to use `@JsonIgnore` on bidirectional relationships to prevent infinite recursion during JSON serialization.

7.1 User Service Entities (`user-service`)

7.1.1 User Entity

Table: `users`

Field	Type	Constraints	Notes
`id`	`Long`	PK, auto-generated	
`name`	`String`	not null	
`email`	`String`	not null, unique	
`password`	`String`	not null	
`phone`	`String`	not null, unique	
`role`	`ENUM`[a]	not null	`"RIDER"` or `"ADMIN"`
`status`	`ENUM`	not null, default ACTIVE	`ACTIVE / DEACTIVATED`
`preferences`	`Map<String, Object>`	JSONB	See below
`createdAt`	`LocalDateTime`	not null	Set on creation
`savedAddresses`	`List<SavedAddress>`	–	Relational Attribute

[a]search about how you would save an enum inside SQL and what are the necessary configurations that you have to do for JPA

JSONB — `preferences`: Language, notification settings (email/sms booleans), default payment method.

Relationships: `@OneToMany` with to `SavedAddress` where the `User` is the inverse side and `SavedAddress` is the owning side.

7.1.2 SavedAddress Entity

Table: `saved_addresses`

Field	Type	Constraints	Notes
`id`	`Long`	PK, auto-generated	
`label`	`String`	not null	e.g., “Home”, “Work”
`address`	`String`	not null	Full text address
`latitude`	`Double`	not null	
`longitude`	`Double`	not null	
`isDefault`	`Boolean`	default false	
`metadata`	`Map<String, Object>`	JSONB	See below
`createdAt`	`LocalDateTime`	not null	
`user`	`User`	–	Relational Attribute

JSONB — `metadata`: Delivery instructions, gate code, landmark.

Relationships: `@ManyToOne` to `User` Where the `User` is the inverse side and `SavedAddress` is the owner side.

7.2 Driver Service Entities (`driver-service`)

7.2.1 Driver Entity

Table: `drivers`

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
name	String	not null	
email	String	not null, unique	
phone	String	not null, unique	
licenseNumber	String	not null, unique	
status	ENUM[a]	not null	AVAILABLE / BUSY / OFFLINE
rating	Double	default 0.0	Running average
totalRatings	Integer	default 0	
vehicleDetails	Map<String, Object>	JSONB	See below
createdAt	LocalDateTime	not null	
driverDocuments	List<DriverDocument>	–	Relational Attribute

[a]search about how you would save an enum ***as an ENUM*** inside SQL and what are the necessary configurations that you have to do for JPA

JSONB — `vehicleDetails`**:** Make, model, year, color, license plate, vehicle type (SEDAN, SUV, etc.).

Relationships: `@OneToMany` to `DriverDocument` where the `Driver` is the inverse side and the `DriverDocument` is the owner Side.

7.2.2 DriverDocument Entity

Table: `driver_documents`

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
type	ENUM[a]	not null	LICENSE / INSURANCE / REGISTRATION
documentUrl	String	not null	URL or path to document [b]
expiryDate	LocalDate	not null	
verified	Boolean	default false	
metadata	Map<String, Object>	JSONB	See below
uploadedAt	LocalDateTime	not null	
driver	Driver	–	Relational Attribute

[a]search about how you would save an enum ***as an ENUM*** inside SQL and what are the necessary configurations that you have to do for JPA

[b]doesn't have to be an actual URL

JSONB — `metadata`**:** Issuing authority, document number, verification notes, rejection reason.

Relationships: `@ManyToOne` to `Driver` where the `Driver` is the inverse side and the `DriverDocument` is the owner Side.

7.3 Ride Service Entities (`ride-service`)

7.3.1 Ride Entity

Table: `rides`

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
userId	Long	not null	FK reference (not JPA-managed)
driverId	Long	nullable	FK reference (assigned later)
pickupLatitude	Double	not null	
pickupLongitude	Double	not null	
dropoffLatitude	Double	not null	
dropoffLongitude	Double	not null	
status	ENUM[a]	not null	See values below
fare	Double	nullable	Set when calculated
metadata	Map<String, Object>	JSONB	See below
requestedAt	LocalDateTime	not null	
completedAt	LocalDateTime	nullable	Set when completed
rideStops	List<RideStop>	–	Relational Attribute

[a]search about how you would save an enum ***as an ENUM*** inside SQL and what are the necessary configurations that you have to do for JPA

Status values: REQUESTED, ACCEPTED, IN_PROGRESS, COMPLETED, CANCELLED.

JSONB — metadata: Surge multiplier, estimated distance (km), estimated duration (minutes), special requests text, ride type (STANDARD, PREMIUM, etc.).

Note: userId and driverId are plain Long columns, **not** JPA @ManyToOne relationships (User and Driver live in different services).

Relationships: @OneToMany to RideStop where the Ride is the inverse side and the RideStop is the owning side.

7.3.2 RideStop Entity

Table: ride_stops

A ride can have multiple intermediate stops (e.g., picking up a friend, dropping off a package). Each stop has an order within the ride.

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
stopOrder	Integer	not null	1, 2, 3...
latitude	Double	not null	
longitude	Double	not null	
address	String	not null	
status	ENUM[a]	not null	PENDING / REACHED / SKIPPED
metadata	Map<String, Object>	JSONB	See below
ride	RideStop	–	Relational Attribute

[a]search about how you would save an enum ***as an ENUM*** inside SQL and what are the necessary configurations that you have to do for JPA

JSONB — metadata: Estimated arrival time, actual arrival time, wait time (minutes), contact name, contact phone.

Relationships: @ManyToOne to Ride where the Ride is the inverse side and the RideStop is the owning side.

7.4 Location Service Entities (location-service)

7.4.1 Location Entity

Table: locations

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
driverId	Long	not null	FK reference (not JPA-managed)
latitude	Double	not null	
longitude	Double	not null	
timestamp	LocalDateTime	not null	
metadata	Map<String, Object>	JSONB	See below

JSONB — `metadata`**:** Heading (degrees), speed (km/h), GPS accuracy (meters).

Relationships: None within this service. `driverId` is a plain `Long` referencing the `drivers` table in the shared database via native SQL.

7.5 Payment Service Entities (`payment-service`)

This service demonstrates the **join entity pattern** for many-to-many relationships with extra columns (like the `OrderItem` pattern from Lab 4).

7.5.1 Payment Entity

Table: `payments`

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
rideId	Long	not null	FK reference (not JPA-managed)
userId	Long	not null	FK reference (not JPA-managed)
amount	Double	not null	
method	ENUM[a]	not null	CREDIT_CARD / CASH / WALLET
status	ENUM	not null	See below
transactionDetails	Map<String, Object>	JSONB	See below
createdAt	LocalDateTime	not null	
paymentCoupons	List<PaymentCoupon>	–	Relational Attribute

[a] search about how you would save an enum ***as an ENUM*** inside SQL and what are the necessary configurations that you have to do for JPA

Status values: `PENDING`, `COMPLETED`, `FAILED`, `REFUNDED`.

JSONB — `transactionDetails`**:** Gateway response, card last four digits, receipt URL, failure reason.

Relationships: `@OneToMany` to `PaymentCoupon` where the `Payment` is the inverse side and the `PaymentCoupon` is the owner side.

7.5.2 Coupon Entity

Table: `coupons`

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
code	String	not null, unique	e.g., "WELCOME50"
discountType	ENUM[a]	not null	PERCENTAGE or FIXED
discountValue	Double	not null	e.g., 50.0
maxUses	Integer	not null	
currentUses	Integer	default 0	
expiryDate	LocalDateTime	not null	
active	Boolean	default true	
metadata	Map<String, Object>	JSONB	See below
paymentCoupons	List<PaymentCoupon>	–	Relational Attribute

[a]search about how you would save an enum ***as an ENUM*** inside SQL and what are the necessary configurations that you have to do for JPA

JSONB — `metadata`**:** Eligible ride types, minimum fare requirement, terms and conditions, applicable regions.

Relationships: `@OneToMany` to `PaymentCoupon` where the `Coupon` is the inverse side and the `PaymentCoupon` is the owner side.

7.5.3 PaymentCoupon Entity (Join Entity)

Table: `payment_coupons`

This is a **join entity** linking `Payment` and `Coupon` in a many-to-many relationship with extra columns — the same pattern as `OrderItem` from Lab 4. A payment can use multiple coupons, and a coupon can be used on multiple payments.

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
discountApplied	Double	not null	Actual amount deducted
appliedAt	LocalDateTime	not null	When coupon was applied
payment	Payment	–	Relational Attribute
coupon	Coupon	–	Relational Attribute

Relationships:

- `@ManyToOne` to `Payment` where the `Payment` is the inverse side and the `PaymentCoupon` is the owner side.
- `@ManyToOne` to `Coupon` where the `Coupon` is the inverse side and the `PaymentCoupon` is the owner side.

7.6 Relationship Summary

Service	Relationship	Type	Pattern Tested
User	User ↔ SavedAddress	@OneToMany / @ManyToOne	Bidirectional
Driver	Driver ↔ DriverDocument	@OneToMany / @ManyToOne	Bidirectional
Ride	Ride ↔ RideStop	@OneToMany / @ManyToOne	Bidirectional + ordering [a]
Location	(no JPA relations)	—	Cross-service via native SQL
Payment	Payment ↔ Coupon	Many-to-Many	Join entity (PaymentCoupon)

[a]You will use the ordering in a specific feature from the feature list

7.7 Important: Cross-Service Data Access

Since all services share one PostgreSQL database, any service can query any table using **native SQL**. For example, the Ride Service can `JOIN rides` with `drivers` or `payments` in a native `@Query`. However, each service only defines JPA entity classes and JPA relationships for entities within its *own* module. Cross-table access is always via native SQL, never via JPA relationships. This pattern will be replaced by inter-service API calls in Milestone 3.

8 Repository Layer

Each service defines a `JpaRepository` for **each of its entities**. For services with multiple entities, you need multiple repository interfaces. ***Make sure that you make use of the Repository Layer when interacting with the database and not have the queries at the service layer as the auto grader will test having proper layering logic in your code.*** Repositories include:

- **Naming-convention methods** for simple lookups.
- **Custom `@Query` methods with native SQL** for complex queries, including JOINs against other services' tables in the shared database.
- `@Modifying` + `@Transactional` for UPDATE/DELETE queries.

9 Features

CRUD operations (create, read by ID, read all, update, delete) are the **baseline** for **every entity in every service** and are NOT counted as features. ***However this doesn't mean that you don't need to implement it as it will have test cases and the auto grader will make use of it and will not run if they are not there so*** This means: User Service needs CRUD for both User and SavedAddress, Driver Service for both Driver and DriverDocument, Ride Service for both Ride and RideStop, Location Service for Location, and Payment Service for Payment, Coupon, and PaymentCoupon. Implement all CRUD first before starting features.

Important Note: Do not forget to apply the Layered architecture (having repository, service and controller) as the test case will test that each feature is implemented according to the layered architecture as explained to you in the Labs.

9.1 User Service Features (port 8081)

9.1.1 [S1-F1] Search Users with Filters

Branch: `feat/user/S1-F1/<ID>`
Endpoint: `GET /api/users/search?name={name}&email={email}&role={role}`
Response: List of users matching any non-null filter. Name and email support partial matching (case-insensitive). Return empty list if no matches.
Note: You may use Native SQL query but you will have to find a way to handle null parameters. **Test scenario:**

a) Create 3 users via CRUD: one with name "Ahmed", role "RIDER"; one with name "Sara", role "ADMIN"; one with name "Ahmed Ali", role "RIDER".

b) `GET /api/users/search?name=Ahmed` → should return 2 users ("Ahmed" and "Ahmed Ali").

c) `GET /api/users/search?role=ADMIN` → should return only "Sara".

d) `GET /api/users/search?name=xyz` → should return empty list.

e) `GET /api/users/search?name=ahmed` → should still return 2 (case-insensitive).

9.1.2 [S1-F2] Update User Preferences (JSONB)

Branch: `feat/user/S1-F2/<ID>`
Endpoint: `PUT /api/users/{id}/preferences`
Request body: JSON object of preference key-value pairs.
Behavior: Find user **Merge** the incoming preferences into existing map—do not overwrite. Return updated user. Return HTTP error code 404 response if the user was not found **Test scenario:**

a) Create a user with preferences `{"language":"en","theme":"light"}`.

b) `PUT /api/users/{id}/preferences` with body `{"theme":"dark","currency":"EGP"}`.

c) Verify response has `{"language":"en","theme":"dark","currency":"EGP"}` — "language" kept, "theme" updated, "currency" added.

d) `PUT /api/users/999/preferences` → should return 404.

9.1.3 [S1-F3] Get User Ride Summary (DTO)

Branch: `feat/user/S1-F3/<ID>`
Endpoint: `GET /api/users/{id}/ride-summary`
Response DTO (*`UserRideSummaryDTO`*): `userId`, `name`, `totalRides`, `completedRides`, `cancelledRides`, `totalSpent`, `averageFare`.
Behavior: Find user –***throws 404 if user not found***. Aggregate data from the `rides` table for this user using native SQL JOINs on the shared database. Build and return DTO. [1] **Test scenario:**

a) Create a user. Create 5 rides for that user (via ride-service CRUD): 3 COMPLETED with fares 50, 75, 100; 1 CANCELLED; 1 REQUESTED.

b) `GET /api/users/{id}/ride-summary` → should return: totalRides=5, completedRides=3, cancelledRides=1, totalSpent=225, averageFare=75.

c) Test with a user who has no rides → all values should be 0.

d) Test with non-existent user ID → 404.

9.1.4 [S1-F4] Deactivate User Account

Branch: `feat/user/S1-F4/<ID>`
Endpoint: `PUT /api/users/{id}/deactivate`
Behavior (Transactional): Find user –***throws 404 if user not found***. Check no active rides exist for this user in the `rides` table –***throw 400 if the user has active rides***. Set `status` to `"DEACTIVATED"`. Save.

Test scenario:

a) Create a user. Create a ride with status REQUESTED for that user.

b) `PUT /api/users/{id}/deactivate` → should return 400 (active ride exists).

c) Update the ride status to COMPLETED.

d) `PUT /api/users/{id}/deactivate` → should succeed. Verify user status is now "DEACTIVATED".

[1] Since you will use native SQL here, you can construct the DTO manually from the Object[] that will be returned from the query

9.1.5 [S1-F5] Filter Users by Preference (JSONB Query)

Branch: `feat/user/S1-F5/<ID>`
Endpoint: `GET /api/users/preferences/search?key={key}&value={value}`
Behavior: Validate key/value not blank —***throw (400) if not validated***. Query users where JSONB `preferences` matches key-value pair. Return matching users. **Test scenario:**

a) Create 3 users: one with preferences `{"language":"ar"}`, one with `{"language":"en"}`, one with `{"language":"ar"}`.

b) `GET /api/users/preferences/search?key=language&value=ar` → should return 2 users.

c) `GET /api/users/preferences/search?key=language&value=fr` → empty list.

d) `GET /api/users/preferences/search?key=&value=ar` → should return 400.

9.1.6 [S1-F6] Top Riders by Spending (Report DTO)

Branch: `feat/user/S1-F6/<ID>`
Endpoint: `GET /api/users/reports/top-riders?startDate={date}&endDate={date}&limit={n}`
Response DTO (`TopRiderDTO`): `userId`, `name`, `totalSpent`, `rideCount`.
Behavior: The feature means show me the top N riders who spent the most money on rides in a given time period. Validate dates –***throws 400 if not valid (if start after end)***. Build the query and construct a DTO from the result of the query in the same way as you did in S1-F3. **Test scenario:**

a) Create 3 users. Create completed rides with fares: User A = 300 total, User B = 500 total, User C = 100 total, all within March 2026.

b) `GET /api/users/reports/top-riders?startDate=2026-03-01&endDate=2026-03-31&limit=2` → should return User B first, then User A.

c) Test with dates where no rides exist → empty list.

d) Test with startDate after endDate → 400.

9.1.7 [S1-F7] Set Default Saved Address (Transactional + Relationship)

Branch: `feat/user/S1-F7/<ID>`
Endpoint: `PUT /api/users/{userId}/addresses/{addressId}/default`
Behavior (Transactional): Find user (404). Find the SavedAddress by ID and verify it belongs to this user —***throw 404 if user not found or the address not found and 400 if the saved address doesn't belong to that user***. Set `isDefault = false` on all of the user's existing addresses. Set `isDefault = true` on the target address. Return the updated user with their addresses. Make sure that cascading is working on the JPA relation **Test scenario:**

a) Create a user with 3 saved addresses. Set address #2 as default.

b) `PUT /api/users/{userId}/addresses/{addr3Id}/default` → verify address #3 is now default and #2 is no longer default.

c) Test with non-existent user → 404.

d) Test with an address ID that belongs to a different user → 400.

9.1.8 [S1-F8] Get User Profile with Addresses (Relationship DTO)

Branch: `feat/user/S1-F8/<ID>`
Endpoint: `GET /api/users/{id}/profile`
Response DTO (`UserProfileDTO`): `userId`, `name`, `email`, `phone`, `preferences` (JSONB), `savedAddresses` (list of address objects with label, address, lat/lng, isDefault, metadata), `totalAddresses` (count).
Behavior: Find user with their saved addresses —***throws (404 if user not found)***. Build the DTO from the user entity and its address collection. The DTO must include the full list of addresses, not just IDs. **Test scenario:**

a) Create a user with preferences and 3 saved addresses (one marked default).

b) `GET /api/users/{id}/profile` → verify DTO contains user info, preferences, full address list with all fields, and totalAddresses = 3.

c) Create a user with no addresses → verify savedAddresses is empty list and totalAddresses = 0.

d) Test with non-existent user → 404.

9.1.9 [S1-F9] Find Users by Language Preference with Minimum Rides (JSONB + SQL)

Branch: `feat/user/S1-F9/<ID>`
Endpoint: `GET /api/users/preferences/language?lang={lang}&minRides={n}`
Behavior: This feature is as if you want to find me all users who speak language x and have completed at least n rides. Validate lang not blank –***throws 400 if blank***. **Test scenario:**

a) Create 3 users: User A (language=ar, 5 completed rides), User B (language=ar, 2 completed rides), User C (language=en, 10 completed rides).

b) `GET /api/users/preferences/language?lang=ar&minRides=3` → should return only User A.

c) `GET /api/users/preferences/language?lang=ar&minRides=1` → should return User A and User B.

d) `GET /api/users/preferences/language?lang=&minRides=1` → 400.

9.2 Driver Service Features (port 8082)

9.2.1 [S2-F1] Search Drivers by Status and Rating Range

Branch: `feat/driver/S2-F1/<ID>`
Endpoint: `GET /api/drivers/search?status={s}&minRating={r}&maxRating={r}`
Behavior: Validate range —***throws 400 if not valide (for example if min>max if min > max)***. Returns a list of the users that matches the search criteria ordered by rating descending. **Test scenario:**

a) Create 3 drivers: ratings 4.5 (AVAILABLE), 3.8 (BUSY), 4.9 (AVAILABLE).

b) `GET /api/drivers/search?status=AVAILABLE&minRating=4.0&maxRating=5.0` → should return 2 drivers, ordered 4.9 first.

c) `GET /api/drivers/search?minRating=3.0&maxRating=4.0` → returns the 3.8 driver (no status filter).

d) `GET /api/drivers/search?minRating=5.0&maxRating=3.0` → 400.

9.2.2 [S2-F2] Update Vehicle Details (JSONB Partial Update)

Branch: `feat/driver/S2-F2/<ID>`
Endpoint: `PUT /api/drivers/{id}/vehicle`
Request body: JSON object of vehicle fields to update.
Behavior: Find driver –***throws 404 if driver not found***. Merge incoming fields into existing `vehicleDetails` map meaning if different attributes were sent in the json body, then add it to the existing one, else then you will update it. For example, if the driver's current vehicleDetails is:

```
{"make": "Toyota", "model": "Camry", "year": 2022, "color": "White", "licensePlate": "ABC-1234"}
```

and the request body sends:

```
{"color": "Black", "licensePlate": "XYZ-9999"}
```

The result should be:

```
{"make": "Toyota", "model": "Camry", "year": 2022, "color": "Black", "licensePlate": "XYZ-9999"}
```

save the updated data and return the updated user record.

Test scenario:

a) Create a driver with vehicleDetails `{"make":"Toyota","color":"White","year":2022}`.

b) `PUT /api/drivers/{id}/vehicle` with body `{"color":"Black","seats":4}`.

c) Verify response: make=Toyota (kept), color=Black (updated), year=2022 (kept), seats=4 (added).

d) Test with non-existent driver → 404.

9.2.3 [S2-F3] Get Driver Earnings Summary (DTO)

Branch: `feat/driver/S2-F3/<ID>`
Endpoint: `GET /api/drivers/{id}/earnings?startDate={date}&endDate={date}`
Response DTO (`DriverEarningsDTO`)**:** `driverId`, `name`, `totalRides`, `totalEarnings`, `averageFare`.
Behavior: Find driver –***throws 404 if driver not found***. Aggregate on `rides` table for this driver in date range and return the required DTO by building it the same way as mentioned before. **Test scenario:**

a) Create a driver. Create 5 completed rides for that driver in March with fares 50, 60, 70, 80, 90.

b) `GET /api/drivers/{id}/earnings?startDate=2026-03-01&endDate=2026-03-31` → totalRides=5, totalEarnings=350, averageFare=70.

c) Test with date range where no rides exist → all zeroes.

d) Test with non-existent driver → 404.

9.2.4 [S2-F4] Update Driver Availability (Transactional)

Branch: `feat/driver/S2-F4/<ID>`
Endpoint: `PUT /api/drivers/{id}/availability`
Request body: `status`.
Behavior (Transactional): Find driver –***throws 404 if driver not found***. If going OFFLINE, check no active rides for this driver in `rides` table (native SQL)—throw 400 if found (if found active rides while we want to set it to OFFLINE). Update status and save the updated driver. No need to return it just ***return status code 200*** to indicate the update was done successfully. **Test scenario:**

a) Create a driver (AVAILABLE). Assign them to a ride (status becomes BUSY via S3-F2).

b) `PUT /api/drivers/{id}/availability` with `{"status":"OFFLINE"}` → 400 (active ride).

c) Complete the ride (via S3-F4). Now try OFFLINE again → 200.

d) Verify driver status is OFFLINE.

9.2.5 [S2-F5] Filter Drivers by Vehicle Type (JSONB)

Branch: `feat/driver/S2-F5/<ID>`
Endpoint: `GET /api/drivers/vehicle-type?type={type}&status={status}`
Behavior: Filter drivers by vehicleType and optional status (meaning that the status is not required to perform the filtering and the status here means if the driver is AVAILABLE, BUSY, or OFFLINE.).
Test scenario:

a) Create 3 drivers: one SEDAN/AVAILABLE, one SUV/AVAILABLE, one SEDAN/OFFLINE.

b) `GET /api/drivers/vehicle-type?type=SEDAN&status=AVAILABLE` → returns 1 driver.

c) `GET /api/drivers/vehicle-type?type=SEDAN` → returns 2 drivers (no status filter).

d) `GET /api/drivers/vehicle-type?type=TRUCK` → empty list.

9.2.6 [S2-F6] Top Rated Drivers Report (DTO)

Branch: `feat/driver/S2-F6/<ID>`
Endpoint: `GET /api/drivers/reports/top-rated?limit={n}`
Response DTO (`TopDriverDTO`): `driverId`, `name`, `rating`, `totalRides`.
Behavior: Get the top rated (n) drivers and return them as TopDriverDTO. **Test scenario:**

a) Create 3 drivers with ratings 4.9, 4.5, 4.2. Create some completed rides for each.

b) `GET /api/drivers/reports/top-rated?limit=2` → returns 2 drivers, 4.9 first.

c) `GET /api/drivers/reports/top-rated?limit=10` → returns all 3.

9.2.7 [S2-F7] Rate a Driver After Ride (Transactional)

Branch: `feat/driver/S2-F7/<ID>`
Endpoint: `POST /api/drivers/{id}/rate`
Request body: `rideId` and `rating` (1–5).
Behavior (Transactional): Find driver –***throws 404 if driver not found***. Verify ride exists and belongs to this driver and is COMPLETED —***throws 400 if the ride doesn't belong to the driver or it's not completed or the rate is not between 1 and 5 and throws 404 if the ride is not found***. Validate rating 1–5. Recalculate running average. Update driver and Save. Return status ***code 200*** if the rating was done successfully **Test scenario:**

a) Create a driver (rating=0, totalRatings=0). Create a COMPLETED ride assigned to this driver.

b) `POST /api/drivers/{id}/rate` with `{"rideId":1,"rating":5}` → 200. Verify rating=5.0, totalRatings=1.

c) Rate again with a different completed ride, rating=3 → verify rating=4.0, totalRatings=2.

d) Test with rating=6 → 400. Test with non-completed ride → 400. Test with ride belonging to another driver → 400.

9.2.8 [S2-F8] Verify Driver Document (Transactional + Relationship)

Branch: `feat/driver/S2-F8/<ID>`
Endpoint: `PUT /api/drivers/{driverId}/documents/{documentId}/verify`
Request body: ***verifiedBy***.
Behavior (Transactional): Find driver –***throws 404 if not found***. Find the DriverDocument by ID –***throws 404 if not found*** and verify it belongs to this driver –***throws 400 if it doesn't belong to the specified driver***. Check the document is not expired (`expiryDate` is in the future) —***throws 400 if expired***. Set `verified = true`. Update the document's JSONB `metadata` to add `verifiedAt` timestamp and `verifiedBy` (from request body). In addition, verify the verifiedBy that is the userID being sent in the request body is an actual ***Admin*** User –(***throw 403 if not***). Return the updated driver with all documents. **Test scenario:**

a) Create a driver. Add a DriverDocument with expiryDate in the future, verified=false. Create an ADMIN user.

b) `PUT /api/drivers/{driverId}/documents/{docId}/verify` with `{"verifiedBy":3}` → verify document is now verified=true and metadata has verifiedAt and verifiedBy.

c) Test with expired document → 400. Test with non-ADMIN verifier → 403.

d) Test with document that belongs to different driver → 400.

9.2.9 [S2-F9] Get Drivers with Expired Documents (Relationship + Report DTO)

Branch: `feat/driver/S2-F9/<ID>`
Endpoint: `GET /api/drivers/documents/expired`
Response DTO (`DriverDocumentAlertDTO`): `driverId`, `driverName`, `driverStatus`, `expiredDocuments` (list of documents where `expiryDate < now`), `expiredCount`.
Behavior: Query all drivers that have at least one expired document. For each matching driver, load their expired documents. Build list of DTOs. Only include drivers with that matches the criteria (having expired documents). **Test scenario:**

a) Create 3 drivers. Add documents: Driver A has 1 expired + 1 valid, Driver B has 2 valid, Driver C has 2 expired.

b) `GET /api/drivers/documents/expired` → returns 2 DTOs (Driver A with expiredCount=1, Driver C with expiredCount=2). Driver B not included.

c) Test with no expired documents in DB → empty list.

9.3 Ride Service Features (port 8083)

9.3.1 [S3-F1] Get Rides by Status and Date Range

Branch: `feat/ride/S3-F1/<ID>`
Endpoint: `GET /api/rides/search?status={s}&startDate={d}&endDate={d}`
Behavior: Query with date range on `requestedAt` and optional status of the rides. Order by most recent the results by the most recent till the oldest. **Test scenario:**

a) Create 5 rides: 2 COMPLETED in March, 1 REQUESTED in March, 2 COMPLETED in February.

b) `GET /api/rides/search?status=COMPLETED&startDate=2026-03-01&endDate=2026-03-31` → returns 2 rides, most recent first.

c) `GET /api/rides/search?startDate=2026-03-01&endDate=2026-03-31` → returns 3 (no status filter).

9.3.2 [S3-F2] Assign Driver to Ride (Transactional)

Branch: `feat/ride/S3-F2/<ID>`
Endpoint: `PUT /api/rides/{rideId}/assign?driverId={driverId}`
Behavior (Transactional): Find ride –***throws 404 if driver not found***. Validate ride status is REQUESTED (–***throws 400 if not valid***—can only assign a driver to a newly requested ride). Verify the driver exists and has status AVAILABLE in (***throws 404 if driver not found, and 400 if not available***). Set the ride's `driverId` to the given driver. Set ride status to ACCEPTED. Update the driver's status to BUSY. Save ride. Return the updated ride. **Test scenario:**

a) Create a REQUESTED ride (no driver). Create an AVAILABLE driver.

b) `PUT /api/rides/{rideId}/assign?driverId={driverId}` → ride status=ACCEPTED, driverId set, driver status=BUSY.

c) Try assigning again → 400 (ride not REQUESTED anymore).

d) Try assigning a BUSY driver to a new REQUESTED ride → 400.

e) Try with non-existent ride → 404. Non-existent driver → 404.

9.3.3 [S3-F3] Get Fare Estimate (DTO)

Branch: `feat/ride/S3-F3/<ID>`
Endpoint: `POST /api/rides/estimate`

Request body: `pickupLatitude`, `pickupLongitude`, `dropoffLatitude`, `dropoffLongitude`.

Response DTO (`FareEstimateDTO`)**:** `estimatedDistance`, `estimatedDuration`, `estimatedFare`, `surgeMultiplier`.

Scenario: Before requesting a ride, a rider enters their pickup and dropoff locations and the app shows them an estimated price — just like when Uber/Careem shows "EGP 75–90" before you tap "Request." This endpoint computes that estimate. It does **not** create a ride in the database — it only calculates and returns the result.

Behavior — step by step:

a) **Calculate distance (km):** Compute the straight-line distance between the pickup and dropoff points. Since the coordinates are in degrees (latitude/longitude), use the Euclidean distance formula and multiply by 111 to convert degrees to approximate kilometers (1 degree ≈ 111 km):

$$\texttt{distance = sqrt((dropoffLat - pickupLat)}^2\texttt{ + (dropoffLon - pickupLon)}^2\texttt{) * 111}$$

For example, if the pickup is at (30.044, 31.235) and dropoff is at (30.100, 31.300):

$$\texttt{distance = sqrt((30.100 - 30.044)}^2\texttt{ + (31.300 - 31.235)}^2\texttt{) * 111} \approx \texttt{9.2 km}$$

b) **Estimate duration (minutes):** Assume an average driving speed of 40 km/h and convert to minutes:

```
duration = (distance / 40) * 60
```

For the 9.2 km example: `(9.2 / 40) * 60` ≈ `14 minutes`.

c) **Determine surge multiplier (demand-based pricing):** This simulates how ride-hailing apps increase prices during high demand. The idea: if many rides are happening near the rider's pickup location right now, the area is "busy" and the fare goes up.

Write a **native SQL query** that counts how many rides in the `rides` table are currently active (status IN `REQUESTED`, `ACCEPTED`, `IN_PROGRESS`) and have a pickup location close to the requested pickup point. "Close" means the ride's `pickupLatitude` is within ±0.01 of the requested latitude, AND the ride's `pickupLongitude` is within ±0.01 of the requested longitude. This defines a roughly 1.1 km × 1.1 km square around the pickup point.

For example, if the pickup is at (30.044, 31.235), the query counts rides where:

```
pickupLatitude BETWEEN 30.034 AND 30.054
AND pickupLongitude BETWEEN 31.225 AND 31.245
AND status IN ('REQUESTED', 'ACCEPTED', 'IN_PROGRESS')
```

Based on the count, apply the following surge tiers:

- 0–10 active rides → `surgeMultiplier = 1.0` (normal pricing)
- 11–20 active rides → `surgeMultiplier = 1.5` (moderate demand)
- More than 20 active rides → `surgeMultiplier = 2.0` (high demand)

d) **Calculate fare:** Using a base fare of 15.0 EGP per km:

```
fare = 15.0 * distance * surgeMultiplier
```

For the 9.2 km example with no surge: `15.0 * 9.2 * 1.0 = 138.0 EGP`.
Same ride during high demand: `15.0 * 9.2 * 2.0 = 276.0 EGP`.

e) **Build and return the DTO** containing all four values: `estimatedDistance`, `estimatedDuration`, `estimatedFare`, `surgeMultiplier`.

Test scenario:

a) `POST /api/rides/estimate` with pickup (30.044, 31.235) and dropoff (30.100, 31.300).

b) Verify distance ≈ 9.2 km, duration ≈ 14 min, surgeMultiplier = 1.0 (no active rides nearby), fare ≈ 138 EGP.

c) Create 15 REQUESTED rides near the same pickup area. Call estimate again → surgeMultiplier should be 1.5.

d) Verify no ride was created in the database (this endpoint is read-only).

9.3.4 [S3-F4] Complete Ride (Transactional Multi-Step)

Branch: `feat/ride/S3-F4/<ID>`
Endpoint: `PUT /api/rides/{id}/complete`
Behavior (Transactional): Find ride –***throws 404 if driver not found***. Validate status IN_PROGRESS –***throws 400 if not***. Set COMPLETED + completedAt. Calculate fare if unset. Update driver status to AVAILABLE . Create payment record. Save ride. Return the ride after the update. **Test scenario:**

a) Create a ride with status IN_PROGRESS, assigned to a BUSY driver, fare=100.

b) `PUT /api/rides/{id}/complete` → ride status=COMPLETED, completedAt set, driver status=AVAILABLE, a new PENDING payment created with amount=100.

c) Try completing again → 400 (already COMPLETED).

d) Try completing a REQUESTED ride → 400.

9.3.5 [S3-F5] Filter Rides by Metadata Field (JSONB)

Branch: `feat/ride/S3-F5/<ID>`
Endpoint: `GET /api/rides/metadata/search?key={key}&value={value}`
Behavior: Validate key/value –***throws 400 if not valid***. Return matches. **Test scenario:**

a) Create 3 rides: one with metadata `{"rideType":"PREMIUM"}`, two with `{"rideType":"STANDARD"}`.

b) `GET /api/rides/metadata/search?key=rideType&value=PREMIUM` → returns 1 ride.

c) `GET /api/rides/metadata/search?key=&value=x` → 400.

9.3.6 [S3-F6] Ride Analytics by Time Period (Report DTO)

Branch: `feat/ride/S3-F6/<ID>`
Endpoint: `GET /api/rides/analytics?startDate={d}&endDate={d}`
Response DTO (`RideAnalyticsDTO`): `totalRides`, `completedRides`, `cancelledRides`, `totalRevenue`, `averageFare`, `completionRate`.
Behavior: Calculate the required fields given for the rides from the start date till the end date. Build the DTO and return it. **Test scenario:**

a) Create 10 rides in March: 7 COMPLETED (fares: 50,60,70,80,90,100,110), 3 CANCELLED.

b) `GET /api/rides/analytics?startDate=2026-03-01&endDate=2026-03-31` → totalRides=10, completedRides=7, cancelledRides=3, totalRevenue=560, averageFare=80, completionRate=70.0.

9.3.7 [S3-F7] Cancel Ride (Transactional)

Branch: `feat/ride/S3-F7/<ID>`
Endpoint: `PUT /api/rides/{id}/cancel`
Behavior (Transactional): Find ride –***throws 404 if driver not found***. Validate status REQUESTED/ACCEPTED –***throws 400 if not valid***. Set CANCELLED. If driver assigned, update driver status to AVAILABLE (native SQL on `drivers`). Save and ***return status code 200*** if the workflow was done successfully **Test scenario:**

a) Create a REQUESTED ride with a driver assigned (status=ACCEPTED, driver=BUSY).

b) `PUT /api/rides/{id}/cancel` → ride status=CANCELLED, driver status=AVAILABLE.

c) Try cancelling a COMPLETED ride → 400.

d) Try cancelling an IN_PROGRESS ride → 400.

9.3.8 [S3-F8] Add Stops to Existing Ride (Transactional + Relationship)

Branch: `feat/ride/S3-F8/<ID>`
Endpoint: `POST /api/rides/{rideId}/stops`
Request body: A list of stop objects, each with `latitude`, `longitude`, `address`, and `metadata`.
Behavior (Transactional): Find ride –***throws 404 if ride is not found***. Validate ride status is REQUESTED or ACCEPTED (–***throws 400 if not valid***—cannot add stops to in-progress or completed rides). Validate each stop has coordinates and address –***throws 400 if not valid***. Determine the `stopOrder` for each new stop by continuing from the ride's current max stop order (or starting at 1 if no stops exist as in there is was no intermediate stops). Create `RideStop` entities with status = PENDING and add them to the ride. Save the ride. Return the updated ride with all its stops, ordered by `stopOrder`. **Test scenario:**

a) Create a REQUESTED ride with no stops.

b) `POST /api/rides/{rideId}/stops` with 2 stops → verify stops created with stopOrder 1 and 2, status=PENDING.

c) Add 1 more stop → verify stopOrder=3.

d) Try adding stops to a COMPLETED ride → 400.

e) Try adding a stop with missing coordinates → 400.

9.3.9 [S3-F9] Get Ride Details with Stops (Relationship DTO)

Branch: `feat/ride/S3-F9/<ID>`
Endpoint: `GET /api/rides/{rideId}/details`
Response DTO (`RideDetailsDTO`): `rideId`, `userId`, `driverId`, `status`, `fare`, `metadata` (JSONB), `stops` (list of stop objects ordered by `stopOrder`, each with id, stopOrder, address, lat/lng, status, metadata), `totalStops`, `completedStops` (count where status = REACHED).
Behavior: Find ride with its stops loaded (–***throws 404 if ride not found*** if not found). Count stops by status (Completed and not Completed). Build the DTO with the full stop list ordered by `stopOrder` ascending. **Test scenario:**

a) Create a ride with 3 stops: 2 REACHED, 1 PENDING.

b) `GET /api/rides/{rideId}/details` → verify DTO has totalStops=3, completedStops=2, stops ordered by stopOrder.

c) Test with ride that has no stops → totalStops=0, completedStops=0, empty stops list.

d) Test with non-existent ride → 404.

9.4 Location Service Features (port 8084)

9.4.1 [S4-F1] Get Latest Location for a Driver

Branch: `feat/location/S4-F1/<ID>`
Endpoint: `GET /api/locations/driver/{driverId}/latest`
Behavior: Verify driver exists (–***throws 404 if driver not found***). Query most recent location. ***throws 404 if none***. **Test scenario:**

a) Create a driver. Post 3 location updates with different timestamps.

b) `GET /api/locations/driver/{driverId}/latest` → should return the most recent location.

c) Test with driver that has no locations → 404.

d) Test with non-existent driver → 404.

9.4.2 [S4-F2] Update Driver Location with Metadata (JSONB)

Branch: `feat/location/S4-F2/<ID>`
Endpoint: `POST /api/locations/driver/{driverId}`
Request body: latitude, longitude, metadata map.
Behavior: Verify driver exists –***throws 404 if driver not found***. Create Location with timestamp = now. Save. Return ***status code 201***. **Test scenario:**

a) Create a driver.

b) `POST /api/locations/driver/{driverId}` with latitude=30.044, longitude=31.235, metadata=`{"speed":45.2}` → 201.

c) Verify the saved location has a timestamp and the metadata.

d) Test with non-existent driver → 404.

9.4.3 [S4-F3] Find Nearby Available Drivers (DTO with Distance)

Branch: `feat/location/S4-F3/<ID>`
Endpoint: `GET /api/locations/nearby?lat={lat}&lon={lon}&radiusKm={r}`
Response DTO (`NearbyDriverDTO`): `driverId`, `driverName`, `latitude`, `longitude`, `distanceKm`.
Behavior: Based on latest location per driver. Filter AVAILABLE, calculate distance (euclidean distance[2] * 111), filter by radius, sort ascending. Map to DTO and return it. **Test scenario:**

a) Create 3 AVAILABLE drivers. Post locations: Driver A at (30.04, 31.23), Driver B at (30.05, 31.24), Driver C at (31.00, 32.00) (far away).

b) `GET /api/locations/nearby?lat=30.044&lon=31.235&radiusKm=5` → returns Driver A and B (sorted by distance), not C.

c) Test with radiusKm=0.1 → might return only the closest driver.

9.4.4 [S4-F4] Batch Location Update (Transactional)

Branch: `feat/location/S4-F4/<ID>`
Endpoint: `POST /api/locations/batch`
Request body: `driverId` and list of location objects.
Behavior (Transactional): Verify driver –***throws 404 if any driver not found***. Validate coordinates –***throws 400 if not valid, like Latitude must be between -90 and 90 Longitude must be between -180 and 180***. Save all with incrementing timestamps. Return ***status code 201 with count***. **Test scenario:**

a) Create a driver.

b) `POST /api/locations/batch` with driverId and 3 location objects → 201 with count=3.

c) Verify all 3 locations are saved with incrementing timestamps.

d) Test with invalid coordinates (latitude=999) → 400.

e) Test with non-existent driver → 404.

9.4.5 [S4-F5] Filter Locations by Metadata (JSONB Query)

Branch: `feat/location/S4-F5/<ID>`
Endpoint: `GET /api/locations/metadata/search?key={k}&operator={op}&value={v}`
Operators: `eq`, `gt`, `lt`.
Behavior: Validate operator –***throws 400 if not valid***. For gt/lt cast JSONB to numeric. Return matches. **Test scenario:**

a) Create locations with metadata: speed=40, speed=60, speed=80.

b) `GET /api/locations/metadata/search?key=speed&operator=gt&value=50` → returns 2 locations (60 and 80).

c) `GET /api/locations/metadata/search?key=speed&operator=eq&value=40` → returns 1.

d) `GET /api/locations/metadata/search?key=speed&operator=xyz&value=50` → 400.

[2] euclidean distance = $\text{sqrt}((x2 - x1)^2 + (y2 - y1)^2)$

9.4.6 [S4-F6] Get Locations in Date Range

Branch: `feat/location/S4-F6/<ID>`
Endpoint: `GET /api/locations/history?startDate={d}&endDate={d}&driverId={id}`
Behavior: Query all locations within the date range on `timestamp` with optional driverId filter. Order by timestamp ascending.

Test scenario:

a) Create 2 drivers. Post 5 locations across both: 3 in March (2 for driver A, 1 for driver B), 2 in February.

b) `GET /api/locations/history?startDate=2026-03-01&endDate=2026-03-31` → returns 3 locations ordered by timestamp ascending.

c) `GET /api/locations/history?startDate=2026-03-01&endDate=2026-03-31&driverId={driverAId}` → returns 2.

d) Test with date range where no locations exist → empty list.

9.4.7 [S4-F7] Purge Old Location Data (Transactional)

Branch: `feat/location/S4-F7/<ID>`
Endpoint: `DELETE /api/locations/purge?olderThanDays={n}`
Behavior (Transactional): Calculate cutoff (if the request says olderThanDays=30, then cutoff = today minus 30 days) Count then delete via native `@Modifying` query. Return deleted count. **Test scenario:**

a) Create 10 locations: 7 with timestamps 60 days ago, 3 with timestamps today.

b) `DELETE /api/locations/purge?olderThanDays=30` → returns deletedCount=7.

c) Verify only 3 recent locations remain in DB.

9.4.8 [S4-F8] Driver Movement Summary (DTO)

Branch: `feat/location/S4-F8/<ID>`
Endpoint: `GET /api/locations/driver/{driverId}/summary?startDate={d}&endDate={d}`
Response DTO (`DriverMovementSummaryDTO`): `driverId`, `totalLocationPoints`, `averageSpeed`, `maxSpeed`, `firstTimestamp`, `lastTimestamp`.
Behavior: Verify driver –***throws 404 if driver not found***. calculate the above metrics from from JSONB (cast to numeric). Build the DTO and return it. **Test scenario:**

a) Create a driver. Post 5 locations in March with metadata speeds: 30, 40, 50, 60, 0.

b) `GET /api/locations/driver/{driverId}/summary?startDate=2026-03-01&endDate=2026-03-31` → totalLocationPoints=5, averageSpeed=36, maxSpeed=60.

c) Test with non-existent driver → 404.

9.4.9 [S4-F9] Find Stationary Drivers (JSONB + Cross-Table DTO)

Branch: `feat/location/S4-F9/<ID>`
Endpoint: `GET /api/locations/stationary?maxSpeed={s}&sinceMinutes={m}`
Response DTO (`StationaryDriverDTO`): `driverId`, `driverName`, `latitude`, `longitude`, `lastSpeed`, `lastUpdated`.
Behavior: Find drivers whose most recent location entry has a JSONB `speed` value ≤ `maxSpeed` (cast to numeric) and whose `timestamp` is within the last `sinceMinutes` minutes. Use a native SQL query joining `locations` with `drivers`, using a subquery to get only the latest location per driver. Filter by the speed threshold and time window. Map results to DTOs. **Test scenario:**

a) Create 3 drivers (AVAILABLE). Post recent locations: Driver A speed=0, Driver B speed=5, Driver C speed=50.

b) `GET /api/locations/stationary?maxSpeed=10&sinceMinutes=30` → returns Driver A and B (speed ≤ 10).

c) Test with maxSpeed=0 → returns only Driver A.

9.5 Payment Service Features (port 8085)

9.5.1 [S5-F1] Get Payments by Status and Date Range

Branch: `feat/payment/S5-F1/<ID>`
Endpoint: `GET /api/payments/search?status={s}&startDate={d}&endDate={d}`
Behavior: Query with date range on `createdAt` and optional status. return the results with the most recent first. **Test scenario:**

a) Create 5 payments: 2 COMPLETED in March, 1 REFUNDED in March, 2 COMPLETED in February.

b) `GET /api/payments/search?status=COMPLETED&startDate=2026-03-01&endDate=2026-03-31` → returns 2, most recent first.

c) `GET /api/payments/search?startDate=2026-03-01&endDate=2026-03-31` → returns 3 (no status filter).

9.5.2 [S5-F2] Process Refund (Transactional + JSONB Update)

Branch: `feat/payment/S5-F2/<ID>`
Endpoint: `PUT /api/payments/{id}/refund`
Request body: `reason` (string).
Behavior (Transactional): Find payment –***throws 404 if payment not found***. Validate status COMPLETED –***throws 400 if not valid***. Set REFUNDED. Add refundReason and refundedAt to JSONB transactionDetails. Save and return the updated payment. **Test scenario:**

a) Create a COMPLETED payment.

b) `PUT /api/payments/{id}/refund` with `{"reason":"driver no show"}` → status=REFUNDED, transactionDetails has refundReason and refundedAt.

c) Try refunding again → 400 (already REFUNDED).

d) Try refunding a FAILED payment → 400.

9.5.3 [S5-F3] User Payment Summary (DTO)

Branch: `feat/payment/S5-F3/<ID>`
Endpoint: `GET /api/payments/user/{userId}/summary`
Response DTO (`UserPaymentSummaryDTO`)**:** `userId`, `totalPayments`, `totalAmount`, `methodBreakdown` (a `Map<String, Double>`).

Behavior — step by step:

a) **Verify user exists:** Check if the user exists –***throws 404 if user not found***.

b) **Query payment data grouped by method:** Query the payements of the user that has status as COMPLETED and then group the results by the method of payment to be able to compute the count and the sum of `amount` per each method.

c) **Build the method breakdown map:** Populate the results into the required DTO and populate a `Map<String, Double>` where each key is the payment method (e.g., `"CREDIT_CARD"`) and the value is the total amount for that method.

d) **Calculate totals:** `totalPayments` is the sum of all counts across all methods. `totalAmount` is the sum of all amounts across all methods. You can compute these by iterating over the map, or by running a separate simpler query.

e) **Build and return the DTO** with `userId`, `totalPayments`, `totalAmount`, and the `methodBreakdown` map.

Test scenario:

a) Create a user. Create 4 COMPLETED payments: 2 via CREDIT_CARD (75, 120), 1 via CASH (50), 1 via WALLET (30).

b) `GET /api/payments/user/{userId}/summary` → totalPayments=4, totalAmount=275, methodBreakdown=`{"CREDIT_CARD":195,"CASH":50,"WALLET":30}`.

c) Test with non-existent user → 404.

9.5.4 [S5-F4] Process Payment for Ride (Transactional)

Branch: `feat/payment/S5-F4/<ID>`
Endpoint: `POST /api/payments/ride/{rideId}`
Request body: `method`, `cardLastFour` (optional).
Behavior (Transactional): Verify ride exists –***throws 404 if not found*** and is COMPLETED –***throws 400 if having different status***. Check no COMPLETED payment exists for this ride – ***throws 400 if not with a message "already paid"***. Update the Payment that was created upon ride completion with status PENDING, populate JSONB. Save. Return ***status code 201*** indicating successful payment. **Test scenario:**

a) Create a COMPLETED ride.

b) `POST /api/payments/ride/{rideId}` with `{"method":"CREDIT_CARD","cardLastFour":"4242"}` → 201. Verify payment created with correct amount and JSONB populated.

c) Try paying again for the same ride → 400 ("already paid").

d) Try paying for a REQUESTED ride → 400.

9.5.5 [S5-F5] Apply Coupon to Payment (Transactional + Join Entity)

Branch: `feat/payment/S5-F5/<ID>`
Endpoint: `POST /api/payments/{paymentId}/coupons/{couponId}`
No request body — both IDs come from the URL path.

Scenario: A rider has just completed a ride. The system created a PENDING payment of 200 EGP. Before the payment is processed, the rider wants to apply a coupon code to get a discount. The system validates the coupon, calculates the discount, and creates a `PaymentCoupon` join entity linking the two — the same pattern as `OrderItem` from Lab 4.

Behavior (Transactional) — step by step:

a) **Find payment** by `paymentId` — *throw 404 if not found.*

b) **Validate payment status:** The payment's `status` must be `PENDING` — *throw 400 with message "cannot apply coupon to a completed/cancelled payment" if status is `COMPLETED` or `REFUNDED`.*

c) **Find coupon** by `couponId` — *throw 404 if not found.*

d) **Validate coupon is usable:**

- Coupon's `active` field must be `true` — *throw 400 if not.*
- Coupon's `expiryDate` must be in the future — *throw 400 if expired.*
- Coupon's `currentUses` must be less than `maxUses` — *throw 400 if limit reached.*

e) **Check for duplicate:** Query the `PaymentCoupon` records for this payment to see if this coupon has already been applied — *throw 400 with message "coupon already applied" if found.*

f) **Calculate the discount amount** based on the coupon's `discountType`:

- If `PERCENTAGE`: `discount = payment.amount * coupon.discountValue / 100`
  Example: 200 EGP payment with a 25% coupon → `200 * 25 / 100 = 50 EGP` discount.
- If `FIXED`: `discount = coupon.discountValue`
  Example: 200 EGP payment with a 30 EGP fixed coupon → 30 EGP discount.

Then **cap the discount**: if the calculated discount exceeds the payment amount, set it to the payment amount (a coupon cannot produce a negative payment).

g) **Create the join entity:** Create a new `PaymentCoupon` with the computed variables.

h) **Update the coupon:** Increment `currentUses` by 1.

i) **Save** the `PaymentCoupon` and the updated coupon.

j) **Return** the updated payment with its list of applied coupons.

Test scenario:

a) Create a PENDING payment (amount=200). Create a coupon: code="SAVE25", PERCENTAGE, discountValue=25, maxUses=3.

b) `POST /api/payments/{paymentId}/coupons/{couponId}` → PaymentCoupon created with discountApplied=50. Coupon currentUses incremented to 1.

c) Apply same coupon again → 400 (already applied).

d) Create a FIXED coupon with discountValue=999. Apply to a 200 EGP PENDING payment → discountApplied should be capped at 200.

e) Test with a COMPLETED payment → 400. Test with expired coupon → 400. Test with inactive coupon → 400.

9.5.6 [S5-F6] Revenue Report by Date Range (Report DTO)

Branch: `feat/payment/S5-F6/<ID>`
Endpoint: `GET /api/payments/reports/revenue?startDate={d}&endDate={d}`
Response DTO (`RevenueReportDTO`): `totalRevenue`, `totalTransactions`, `averagePayment`, `refundedAmount`, `refundCount`.

Scenario: An admin wants to see how much money the platform made in a given period, and how much was lost to refunds.

Behavior — step by step:

a) **Validate dates:** Throw 400 if `startDate` is after `endDate`.

b) **Query payments in date range**

c) **The five metrics and how to calculate each:**

- `totalRevenue` — Sum of `amount` for all payments where `status = 'COMPLETED'`. This is the money the platform actually received.
- `totalTransactions` — Count of payments where `status = 'COMPLETED'`. This is how many successful payments were made.
- `averagePayment` — `totalRevenue / totalTransactions`. The average size of a successful payment. Handle division by zero (if no completed payments, return 0).
- `refundedAmount` — Sum of `amount` for all payments where `status = 'REFUNDED'`. This is how much money was given back to riders.
- `refundCount` — Count of payments where `status = 'REFUNDED'`.

d) **Build and return the DTO** with all five values.

Test scenario:

a) Create payments in March: 5 COMPLETED (amounts: 50,60,70,80,90 = total 350), 2 REFUNDED (amounts: 40,60 = total 100).

b) `GET /api/payments/reports/revenue?startDate=2026-03-01&endDate=2026-03-31` → totalRevenue=350, totalTransactions=5, averagePayment=70, refundedAmount=100, refundCount=2.

c) Test with startDate after endDate → 400.

9.5.7 [S5-F7] Retry Failed Payment (Transactional)

Branch: `feat/payment/S5-F7/<ID>`
Endpoint: `PUT /api/payments/{id}/retry`
No request body — the payment ID comes from the URL path.

Scenario: A payment failed (e.g., card declined, network timeout). The rider wants to try again. Instead of creating a new payment, the system retries the existing one — marking it as successful and recording that it was a retry.

Behavior (Transactional) — step by step:

a) **Find payment** by `id` — *throw 404 if not found.*

b) **Validate status:** The payment's status must be `FAILED`. You can only retry a failed payment — *throw 400 if the status is anything else* (PENDING, COMPLETED, or REFUNDED).

c) **Update status:** Set the payment's status to `COMPLETED`.

d) **Update JSONB `transactionDetails`:** This is where you track the retry history. Two fields to update in the map:

- `retryAttempt` — Read the current value (it may be 0 or a previous number), increment it by 1.
- `gatewayResponse` — Overwrite with the string `"approved"` (simulating that the payment gateway accepted it this time).

So if the JSONB was:

```
{
  "gatewayResponse":"declined",
  "cardLastFour":"4242",
  "retryAttempt":0,
  "failureReason":"insufficient funds"
}
```

After retry it becomes:

```
{
  "gatewayResponse":"approved",
  "cardLastFour":"4242",
  "retryAttempt":1,
  "failureReason":"insufficient funds"
}
```

e) **Save** the payment.

f) **Return** the updated payment entity.

Test scenario:

a) Create a FAILED payment with transactionDetails `{"gatewayResponse":"declined","retryAttempt":0}`.

b) `PUT /api/payments/{id}/retry` → status=COMPLETED, retryAttempt=1, gatewayResponse="approved".

c) Try retrying a COMPLETED payment → 400.

d) Try retrying a REFUNDED payment → 400.

9.5.8 [S5-F8] Get Payment Details with Applied Coupons (Join Entity DTO)

Branch: `feat/payment/S5-F8/<ID>`
Endpoint: `GET /api/payments/{paymentId}/details`
Response DTO (`PaymentDetailsDTO`): `paymentId`, `rideId`, `userId`, `originalAmount`, `method`, `status`, `transactionDetails` (JSONB), `appliedCoupons` (list of objects, each with `couponCode`, `discountType`, `discountApplied`, `appliedAt`), `totalDiscount` (sum of all discounts), `finalAmount` (`originalAmount - totalDiscount`).
Behavior: Find payment with its `PaymentCoupon` entries loaded (–***throws 404 if payment not found***). For each `PaymentCoupon`, access the linked Coupon entity to get the coupon code and discount type. Calculate totalDiscount and finalAmount. Build the DTO and return it. **Test scenario:**

a) Create a payment (amount=200). Apply 2 coupons: one with discountApplied=50, one with discountApplied=30.

b) `GET /api/payments/{paymentId}/details` → originalAmount=200, appliedCoupons has 2 entries with coupon codes and discount info, totalDiscount=80, finalAmount=120.

c) Test with payment that has no coupons → totalDiscount=0, finalAmount=originalAmount.

d) Test with non-existent payment → 404.

9.5.9 [S5-F9] Get Most Used Coupons Report (Join Entity + Aggregation)

Branch: `feat/payment/S5-F9/<ID>`
Endpoint: `GET /api/payments/coupons/top-used?limit={n}`
Response DTO (`CouponUsageDTO`): `couponId`, `code`, `discountType`, `discountValue`, `timesUsed`, `totalDiscountGiven`, `active`, `expired`.

Scenario: A platform manager wants to see which coupons are performing best — which ones are used the most and how much discount they've given out in total. This helps decide whether to extend, deactivate, or create similar promotions.

Behavior — step by step:

a) **Write a query** that searches for the `coupons` inside the `payment_coupons` table. Group the results by coupon. For each coupon, compute:

- `timesUsed` — the coupon's `currentUses` value (how many times it has been applied overall).
- `totalDiscountGiven` — `SUM(payment_coupons.discount_applied)` for all `PaymentCoupon` rows linked to this coupon. This is the total money saved by all riders who used this coupon.

For example, if coupon "WELCOME50" was applied to 3 payments with discounts of 50, 50, and 40 EGP, then `timesUsed = 3` and `totalDiscountGiven = 140.0`.

b) **Order by** `timesUsed` descending (most popular coupons first) and **limit** to `n` results.

c) **Map each of the results to a** `CouponUsageDTO`**:** For each result, populate:

- `couponId`, `code`, `discountType`, `discountValue` — straight from the coupon columns.
- `timesUsed` — from `currentUses`.
- `totalDiscountGiven` — from the SUM computed above.
- `active` — the coupon's `active` field.
- `expired` — a computed boolean: `true` if the coupon's `expiryDate` is before now, `false` otherwise.

d) **Return** the list of DTOs.

Test scenario:

a) Create 3 coupons. Apply coupon A to 5 payments (total discount=250), coupon B to 3 payments (total=90), coupon C to 1 payment (total=20).

b) `GET /api/payments/coupons/top-used?limit=2` → returns coupon A (timesUsed=5, totalDiscountGiven=250) then coupon B (timesUsed=3, totalDiscountGiven=90).

c) Verify "expired" field is true for coupons past their expiryDate.

10 Git Workflow — Feature Development

Every team member follows this workflow for **every feature**:

a) Start from main: `git checkout main && git pull origin main`

b) Create feature branch: `git checkout -b feat/<service>/<feature-ID>/<YOUR-STUDENT-ID>`

c) Implement: repository methods → service logic → controller endpoint → test via Postman.

d) Commit: `git commit -m "feat(<service-name>): <description> (<YOUR-STUDENT-ID>)"` (except only the first commit which is the initialization of the project)

e) Push and create a Pull Request on GitHub.

f) At least 1 teammate reviews and approves.

g) **Merge into main using a regular merge commit ("Create a merge commit" on GitHub). Do NOT use squash merge.** A regular merge preserves the branch name in the merge commit message (e.g., `Merge pull request #12 from feat/user/S1-F1/55-8078`), which the auto-grader uses to verify who implemented which feature.

h) **Do NOT delete the feature branch after merging.** The auto-grader will verify that each feature has a correctly-named branch containing the student ID. If you delete the branch, the grader will not be able to verify it and this may result in deductions in your grade.

11 Dockerization (Phase D)

a) Create a `Dockerfile` inside each service module using `eclipse-temurin:25.0.2_10-jdk`. Copy that service's JAR and expose port 8080.

b) Update the root `docker-compose.yaml` to include all 5 services alongside PostgreSQL. Each service:

- Builds from its own Dockerfile (e.g., `build: ./user-service`)
- Maps its host port to container port 8080 (e.g., `8081:8080`)
- Overrides datasource URL to `jdbc:postgresql://postgres:5432/uberdb`
- Depends on the PostgreSQL service

c) Build: `mvn clean package -DskipTests` from the root.

d) Run: `docker compose up --build`

e) Verify all services respond:

- `localhost:8081/api/users` (User Service)
- `localhost:8082/api/drivers` (Driver Service)
- `localhost:8083/api/rides` (Ride Service)
- `localhost:8084/api/locations` (Location Service)
- `localhost:8085/api/payments` (Payment Service)

f) Git: branch `feat/docker/<ID>`, commit with ID, PR, merge.

12 Work Distribution (Recommended):

Each member implements around 3 features. Phase A (all entities + CRUD for every entity) is done collectively (You can divide it across the sub_team that will implement the service or divide it across all the members equally as you wish).

13 Submission Guidlines

a) Repository must be **private**.

b) `Scalable-Submissions` must be added as a collaborator.

c) `team.json` must be present and valid at the project root.

d) `docker compose up` must start PostgreSQL + all 5 services.

e) Services must respond at `localhost:8081` through `localhost:8085`.

f) Git history must show feature branches merged via PRs with student IDs.

g) Commits must follow `feat(<service>): <desc> (<ID>)` format.

h) Details about the usage of the auto grader and the testing will be sent later

Important:

- Grading is **per member**. A feature only counts for the member whose GitHub username authored the commits on a branch containing their student ID. Features with mismatched or missing IDs receive zero credit. (The **zero** will afect ***both***, the team member and the whole team).

- The only case that the team member will receive a different grade from the rest of the team is if there is no history of commits that shows that this member worked in the milestone (if his/her work was done then the whole team will receive their grade fully and he/she will receive zero. If his/her work was missing from the overall work, the whole team will be deducted for the missing work and he/she will still receive zero).
- It is the responsibility of the scrum master to make sure that everyone is working and the whole requirements are being delivered, if there is an issue with any member(s) then a form will be sent for complains that you can fill it

Deadline: The Milestone deadline is ***Saturday 11/04/2026 at 11:59 PM***

Submission: *Submission form will be sent later to the scrum masters.*

14 Appendix: JSONB Sample Data

Below is one example per service showing what the JSONB columns look like in practice. A full database seeder will be provided separately.

User.preferences:

```
{
  "language": "en",
  "notifications": {"email": true, "sms": false},
  "defaultPaymentMethod": "CREDIT_CARD"
}
```

Driver.vehicleDetails:

```
{
  "make": "Toyota",
  "model": "Camry",
  "year": 2022,
  "color": "White",
  "licensePlate": "ABC-1234",
  "vehicleType": "SEDAN"
}
```

Ride.metadata:

```
{
  "surgeMultiplier": 1.5,
  "estimatedDistance": 12.3,
  "estimatedDuration": 25,
  "specialRequests": "wheelchair accessible",
  "rideType": "STANDARD"
}
```

Location.metadata:

```
{
  "heading": 180.5,
  "speed": 45.2,
  "accuracy": 5.0,
  "batteryLevel": 78
}
```

Payment.transactionDetails:

```
{
  "gatewayResponse": "approved",
  "cardLastFour": "4242",
  "receiptUrl": "https://receipts.example.com/abc",
  "failureReason": null
}
```

15 Appendix: Example Database Tables

The following tables show what your database should look like after seeding some test data. Use these as a reference when testing your CRUD operations and features via Postman. All tables coexist in the same `uberdb` database.

Note: JSONB columns are shown as formatted JSON code blocks below each table for readability. In the actual database, each block is stored as a single JSONB column value in the corresponding row.

15.1 User Service Tables

15.1.1 `users` table

id	name	email	password	phone	role	status	created_at
1	Ahmed Hassan	ahmed@mail.com	pass123	+201011111111	RIDER	ACTIVE	2026-03-01 10:00
2	Sara Mohamed	sara@mail.com	pass456	+201022222222	RIDER	ACTIVE	2026-03-02 14:30
3	Admin User	admin@uber.com	admin789	+201033333333	ADMIN	ACTIVE	2026-03-01 08:00

JSONB column — `preferences`:

`User 1 (Ahmed):`

```
{
  "language": "ar",
  "notifications": {"email": true, "sms": false},
  "defaultPaymentMethod": "CREDIT_CARD"
}
```

`User 2 (Sara):`

```
{
  "language": "en",
  "notifications": {"email": true, "sms": true},
  "defaultPaymentMethod": "WALLET"
}
```

`User 3 (Admin):`

```
{
  "language": "en",
  "notifications": {"email": true, "sms": false}
}
```

15.1.2 `saved_addresses` table

id	user_id	label	address	lat	lon	is_default
1	1	Home	15 Nile St, Zamalek	30.0561	31.2194	true
2	1	Work	42 Smart Village, Giza	30.0709	31.0174	false
3	2	Home	8 Osman Ibn Affan, Heliopolis	30.0887	31.3225	true

JSONB column — `metadata`:

`Address 1 (Ahmed -- Home):`

```
{
  "deliveryInstructions": "Ring doorbell twice",
  "gateCode": "1234",
  "landmark": "Next to Zamalek Club"
}
```

`Address 2 (Ahmed -- Work):`

```
{
  "deliveryInstructions": "Building B, 3rd floor",
  "landmark": "Smart Village main gate"
}
```

`Address 3 (Sara -- Home):`

```
{
  "deliveryInstructions": "Apartment 5A",
  "landmark": "Near Triumph Hotel"
}
```

15.2 Driver Service Tables

15.2.1 `drivers` table

id	name	email	phone	status	rating	total_r	created_at
1	Omar Khaled	omar@mail.com	+201044444444	AVAILABLE	4.8	3	2026-03-01
2	Fatma Ali	fatma@mail.com	+201055555555	BUSY	4.5	2	2026-03-03

(licenseNumber column omitted for space)

JSONB column — `vehicleDetails`:

`Driver 1 (Omar):`

```
{
  "make": "Hyundai",
  "model": "Elantra",
  "year": 2024,
  "color": "White",
  "licensePlate": "       1234",
  "vehicleType": "SEDAN"
}
```

`Driver 2 (Fatma):`

```
{
  "make": "Toyota",
  "model": "Fortuner",
```

```
  "year": 2025,
  "color": "Black",
  "licensePlate": "       5678",
  "vehicleType": "SUV"
}
```

15.2.2 `driver_documents` table

id	driver_id	type	document_url	expiry_date	verified
1	1	LICENSE	/docs/omar-license.pdf	2028-06-15	true
2	1	INSURANCE	/docs/omar-insurance.pdf	2027-01-01	true
3	2	LICENSE	/docs/fatma-license.pdf	2029-03-20	true
4	2	REGISTRATION	/docs/fatma-reg.pdf	2027-09-01	false

(uploadedAt: Doc 1 = 2026-02-15, Doc 2 = 2026-02-20, Doc 3 = 2026-03-01, Doc 4 = 2026-03-05)

JSONB column — `metadata`:

`Document 1 (Omar -- License):`

```
{
  "issuingAuthority": "Cairo Traffic Department",
  "documentNumber": "DL-2024-78901",
  "verificationNotes": "Verified by admin on 2026-03-05"
}
```

`Document 4 (Fatma -- Registration, unverified):`

```
{
  "issuingAuthority": "Giza Traffic Department",
  "documentNumber": "REG-2025-34567",
  "rejectionReason": null
}
```

15.3 Ride Service Tables

15.3.1 `rides` table

id	user_id	driver_id	status	fare	requested_at
1	1	1	COMPLETED	85.00	2026-03-10 08:30
2	2	2	IN_PROGRESS	*null*	2026-03-18 17:00
3	1	*null*	CANCELLED	45.00	2026-03-05 12:00

(pickup/dropoff coordinates and completedAt omitted for space)

JSONB column — `metadata`:

`Ride 1 (Ahmed → Omar, Completed):`

```
{
  "surgeMultiplier": 1.0,
  "estimatedDistance": 18.5,
  "estimatedDuration": 35,
  "specialRequests": "quiet ride please",
  "rideType": "STANDARD"
}
```

`Ride 2 (Sara → Fatma, In Progress):`

```
{
  "surgeMultiplier": 1.5,
  "estimatedDistance": 22.0,
  "estimatedDuration": 45,
  "specialRequests": null,
  "rideType": "PREMIUM"
}
```

Ride 3 (Ahmed, Cancelled):

```
{
  "surgeMultiplier": 1.0,
  "estimatedDistance": 8.0,
  "estimatedDuration": 20,
  "rideType": "STANDARD",
  "cancellationReason": "Driver took too long"
}
```

15.3.2 ride_stops table

id	ride_id	stop_order	address	status
1	1	1	22 Lebanon St, Mohandessin	REACHED
2	1	2	5 Sphinx Square, Giza	REACHED
3	2	1	10 Makram Ebeid, Nasr City	PENDING

(latitude, longitude omitted for space)

JSONB column — metadata:

Stop 1 (Ride 1, Mohandessin -- Reached):

```
{
  "estimatedArrivalTime": "2026-03-10T08:45",
  "actualArrivalTime": "2026-03-10T08:47",
  "waitTime": 3,
  "contactName": "Nour Hassan",
  "contactPhone": "+201066666666"
}
```

Stop 3 (Ride 2, Nasr City -- Pending):

```
{
  "estimatedArrivalTime": "2026-03-18T17:25",
  "actualArrivalTime": null,
  "waitTime": null,
  "contactName": "Ali Mohamed",
  "contactPhone": "+201077777777"
}
```

Reading the data: Ride #1 was Ahmed's morning commute — picked up in Zamalek, stopped in Mohandessin to pick up Nour, then continued to Smart Village. Both stops were reached. Ride #2 is Sara's evening ride currently in progress from Heliopolis to Maadi, with a stop in Nasr City to pick up Ali. Ride #3 was Ahmed's cancelled ride — no driver was assigned before cancellation.

15.4 Location Service Tables

15.4.1 `locations` table

id	driver_id	latitude	longitude	timestamp
1	1	30.0561	31.2194	2026-03-10 08:30
2	1	30.0520	31.2010	2026-03-10 08:45
3	1	30.0709	31.0174	2026-03-10 09:05
4	2	30.0887	31.3225	2026-03-18 17:00
5	2	30.0750	31.3100	2026-03-18 17:15

JSONB column — `metadata`:

`Location 1 (Omar -- pickup at Zamalek):`

```
{
  "heading": 180,
  "speed": 0,
  "gpsAccuracy": 5.2
}
```

`Location 3 (Omar -- arrived at Smart Village):`

```
{
  "heading": 270,
  "speed": 45,
  "gpsAccuracy": 3.8
}
```

`Location 5 (Fatma -- en route):`

```
{
  "heading": 200,
  "speed": 60,
  "gpsAccuracy": 4.1
}
```

Reading the data: Omar's 3 location pings trace his Ride #1 path: Zamalek (pickup) → Mohandessin (stop) → Smart Village (dropoff). Fatma's 2 pings show her current Ride #2 moving from Heliopolis towards Nasr City.

15.5 Payment Service Tables

15.5.1 `payments` table

id	ride_id	user_id	amount	method	status	created_at
1	1	1	85.00	CREDIT_CARD	COMPLETED	2026-03-10 09:10
2	2	2	120.00	WALLET	PENDING	2026-03-18 17:30
3	3	1	45.00	CREDIT_CARD	REFUNDED	2026-03-05 12:15

JSONB column — `transactionDetails`:

`Payment 1 (Ahmed -- Completed):`

```
{
  "gatewayResponse": "approved",
  "cardLastFour": "4242",
  "receiptUrl": "https://pay.example/ride001",
  "failureReason": null
```

```
}
```

Payment 2 (Sara -- Pending):

```
{
  "gatewayResponse": null,
  "walletBalance": 5000,
  "failureReason": null
}
```

Payment 3 (Ahmed -- Refunded):

```
{
  "gatewayResponse": "approved",
  "cardLastFour": "4242",
  "refundReason": "Ride cancelled",
  "refundedAt": "2026-03-05T12:30",
  "failureReason": null
}
```

15.5.2 coupons table

id	code	discount_type	value	max	used	expiry_date	active
1	WELCOME50	PERCENTAGE	50.0	1	1	2026-06-30 23:59	true
2	SAVE20	FIXED	20.0	100	0	2026-12-31 23:59	true
3	FIRST10	PERCENTAGE	10.0	50	50	2026-02-28 23:59	false

(value = discountValue, max = maxUses, used = currentUses)

JSONB column — metadata:

Coupon 1 (WELCOME50):

```
{
  "eligibleRideTypes": ["STANDARD", "PREMIUM"],
  "minimumFare": 30,
  "termsAndConditions": "First ride only, max discount 100 EGP"
}
```

Coupon 2 (SAVE20):

```
{
  "eligibleRideTypes": ["STANDARD"],
  "minimumFare": 50,
  "applicableRegions": ["Cairo", "Giza"]
}
```

Coupon 3 (FIRST10):

```
{
  "eligibleRideTypes": ["STANDARD", "PREMIUM", "POOL"],
  "minimumFare": 20,
  "termsAndConditions": "Launch promotion - first 50 riders"
}
```

15.5.3 `payment_coupons` table (Join Entity)

id	payment_id	coupon_id	discount_applied	applied_at
1	1	1	42.50	2026-03-10 09:08

Reading the data:

- Payment #1 (Ahmed's completed ride, 85 EGP) had coupon WELCOME50 applied — 50% of 85 = 42.50 EGP discount.
- Payment #2 (Sara's in-progress ride, 120 EGP) has no coupons applied yet — it's still PENDING.
- Payment #3 (Ahmed's cancelled ride) was refunded with no coupons.
- Coupon WELCOME50 has `current_uses = 1` because it was used once by Ahmed (single-use coupon, now maxed out).
- Coupon FIRST10 is inactive because it reached its max uses (50/50) and its expiry date has passed.

15.6 How the Tables Connect (Cross-Service)

Since all services share one database, you can trace data across tables:

a) **Ahmed (user 1)** requested **Ride #1** and was matched with **Omar (driver 1)**. The ride went from Zamalek to Smart Village with a stop in Mohandessin to pick up Nour.

b) **Omar's location pings** (locations 1–3) trace the entire route: Zamalek → Mohandessin → Smart Village.

c) After completion, **Payment #1** was charged to Ahmed's credit card (85 EGP) with coupon WELCOME50 applied (saved 42.50 EGP).

d) **Sara (user 2)** is currently in **Ride #2** with **Fatma (driver 2)**, going from Heliopolis to Maadi. Fatma's location pings show her en route. The payment is still PENDING.

e) A native SQL query in the **Ride Service** can JOIN `rides` with `users` (user_id), `drivers` (driver_id), `locations` (driver_id), and `payments` (ride_id) to build a complete ride history DTO.

This is the kind of cross-service data access you will implement using **native SQL JOINs** on the shared database.